

LEVEL 6: Binary Search

1. Objective

Search a sorted book array by repeatedly dividing the search interval into two halves.

2. Data Structure / Algorithm Used

Sorted Array of Structures + Binary Search

3. Problem Statement

Extend the Library Management System by implementing the binary search module. Complete this module independently before integrating it with previous levels.

4. Algorithm / Working

Set low and high → calculate mid → compare key with middle ID → discard one half → repeat.

5. C Program

```
#include <stdio.h>
typedef struct { int id; char title[40]; } Book;

int binarySearch(Book b[], int n, int key) {
    int low=0, high=n-1;
    while(low<=high) {
        int mid=(low+high)/2;
        if(b[mid].id==key) return mid;
        if(key<b[mid].id) high=mid-1;
        else low=mid+1;
    }
    return -1;
}

int main() {
    Book b[]={101,"C Programming"},{103,"OS"},
             {105,"Data Structures"},{110,"DBMS"},{115,"Networks"};
    int pos,key;
    printf("Enter Book ID: "); scanf("%d",&key);
    pos=binarySearch(b,5,key);
    if(pos!=-1) printf("Found: %d - %s\n",b[pos].id,b[pos].title);
    else printf("Book Not Found\n");
    return 0;
}
```

6. Sample Input

Enter Book ID: 110

7. Sample Output

Found: 110 - DBMS

8. Complexity

Iterative Binary Search: $O(\log n)$ time and $O(1)$ extra space. Data must be sorted.

9. Practice Exercises

- Implement recursive Binary Search.
- Count comparisons.
- Explain why sorting is required.

10. Related Viva Questions

- What is the prerequisite for Binary Search?
- What are low, high and mid?
- Why is it $O(\log n)$?
- Can it work on unsorted data?